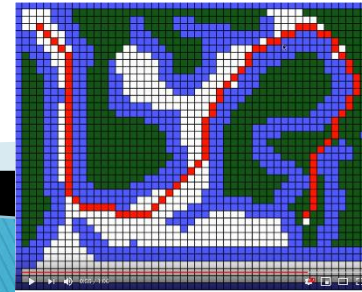


COMP1008 - Fundamentals of AI

Search Strategies

Professor Rong Qu
rong.qu@nottingham.ac.uk



Search – General Search Algorithm

- ▶ Function **General-Search**(p, **QUEUING-FN**) returns a solution or failure
 - nodes = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If nodes is empty then return failure
 - node = Remove-Front(nodes)
 - If Goal-Test[p] on State(node) succeeds then return node
 - nodes = **QUEUING-FN**(nodes, Expand(node, Operators[p]))
 - End
- ▶ End Function

Data structures: (**queue**)

Functions: **QUEUING-FN**

Search – General Search Algorithm

- ▶ Function General-Search(*p*, QUEUING-FN) returns a solution or failure
 - *nodes* = **Make-Queue**(**Make-Node**(Initial-State[*p*]))
 - Loop do
 - If *nodes* is empty then return failure
 - node = Remove-Front(*nodes*)
 - If Goal-Test[*p*] on State(node) succeeds then return node
 - *nodes* = QUEUING-FN(*nodes*, Expand(node, Operators[*p*]))
 - End
- ▶ End Function

Data structures: *nodes*: Queue; node: state;

Functions: QUEUING-FN; **Make-Queue**; **Make-Node**; Goal-test; Operators; Remove-Front;
Expand

Search – General Search Algorithm

- ▶ Function General-Search(p , QUEUING-FN) returns a solution or failure
 - $nodes$ = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If $nodes$ is empty then return failure
 - $node$ = Remove-Front($nodes$)
 - If Goal-Test[p] on State($node$) succeeds then return $node$
 - $nodes$ = QUEUING-FN($nodes$, Expand($node$, Operators[p]))
 - End
- ▶ End Function

Data structures: $nodes$: Queue; $node$: state;

Functions: QUEUING-FN; Make-Queue; Make-Node; Goal-test; Operators; Remove-Front;
Expand

Search – General Search Algorithm

- ▶ Function General-Search(p , QUEUING-FN) returns a solution or failure
 - $nodes$ = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If $nodes$ is empty then return failure
 - $node$ = Remove-Front($nodes$)
 - If Goal-Test[p] on State($node$) succeeds then return $node$
 - $nodes$ = QUEUING-FN($nodes$, Expand($node$, Operators[p]))
 - End
- ▶ End Function

Data structures: $nodes$: Queue; $node$: state;

Functions: QUEUING-FN; Make-Queue; Make-Node; Goal-test; Operators; Remove-Front;
Expand

Search – General Search Algorithm

- ▶ Function General-Search(p, QUEUING-FN) returns a solution or failure
 - *nodes* = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If *nodes* is empty then return failure
 - node = Remove-Front(*nodes*)
 - If **Goal-Test**[p] on State(**node**) succeeds then return node
 - *nodes* = QUEUING-FN(*nodes*, Expand(node, Operators[p]))
 - End
- ▶ End Function

Data structures: *nodes*: Queue; **node**: state;

Functions: QUEUING-FN; Make-Queue; Make-Node; **Goal-test**; Operators; Remove-Front;
Expand

Search – General Search Algorithm

- ▶ Function General-Search(p , QUEUING-FN) returns a solution or failure
 - $nodes$ = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If $nodes$ is empty then return failure
 - $node$ = Remove-Front($nodes$)
 - If Goal-Test[p] on State($node$) succeeds then return $node$
 - $nodes$ = **QUEUING-FN**($nodes$, Expand($node$, Operators[p]))
 - End
- ▶ End Function

Data structures: $nodes$: Queue; $node$: state;

Functions: **QUEUING-FN**; Make-Queue; Make-Node; Goal-test; **Operators**; Remove-Front;
Expand

Breadth First Search (BFS) - Method

- ▶ Expand Root Node First
- ▶ Expand all nodes at level 1 before expanding level 2
OR
- ▶ Expand all nodes at level d before expanding nodes at level $d+1$

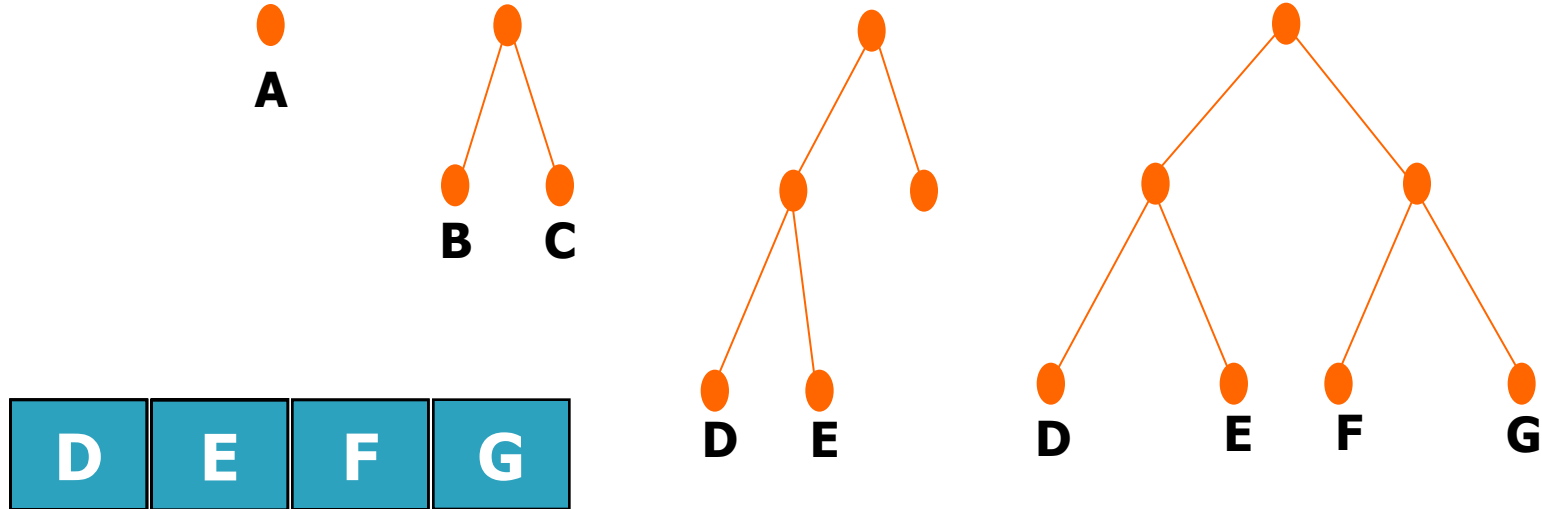
Queuing function: adds nodes to the End of the queue

Function **BFS(problem)** returns a solution or failure

Return **General-Search(problem, ENQUEUE-AT-END)**

- ▶ Function General-Search(p , QUEUING-FN) returns a solution or failure
 - $nodes$ = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If $nodes$ is empty then return failure
 - $node$ = Remove-Front($nodes$)
 - If Goal-Test[p] on State($node$) succeeds then return $node$
 - $nodes$ = QUEUING-FN($nodes$, Expand($node$, Operators[p]))
 - End
 - ▶ End Function

BFS - Implementation



node = **Remove-Front**(**nodes**)

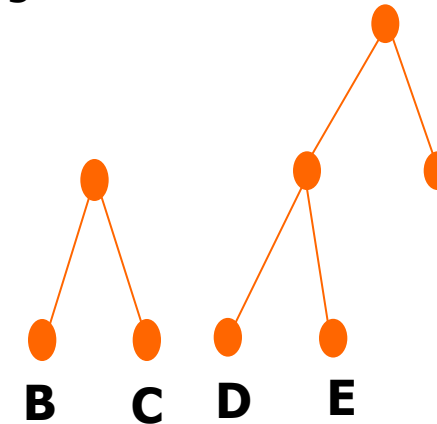
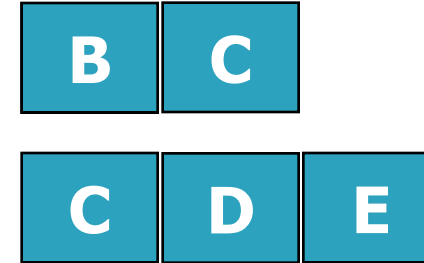
If **Goal-Test**[p] on **State**(**node**) succeeds then return **node**

nodes = **QUEUING-FN**(**nodes**, **Expand**(**node**, **Operators**[p]))

BFS - Implementation

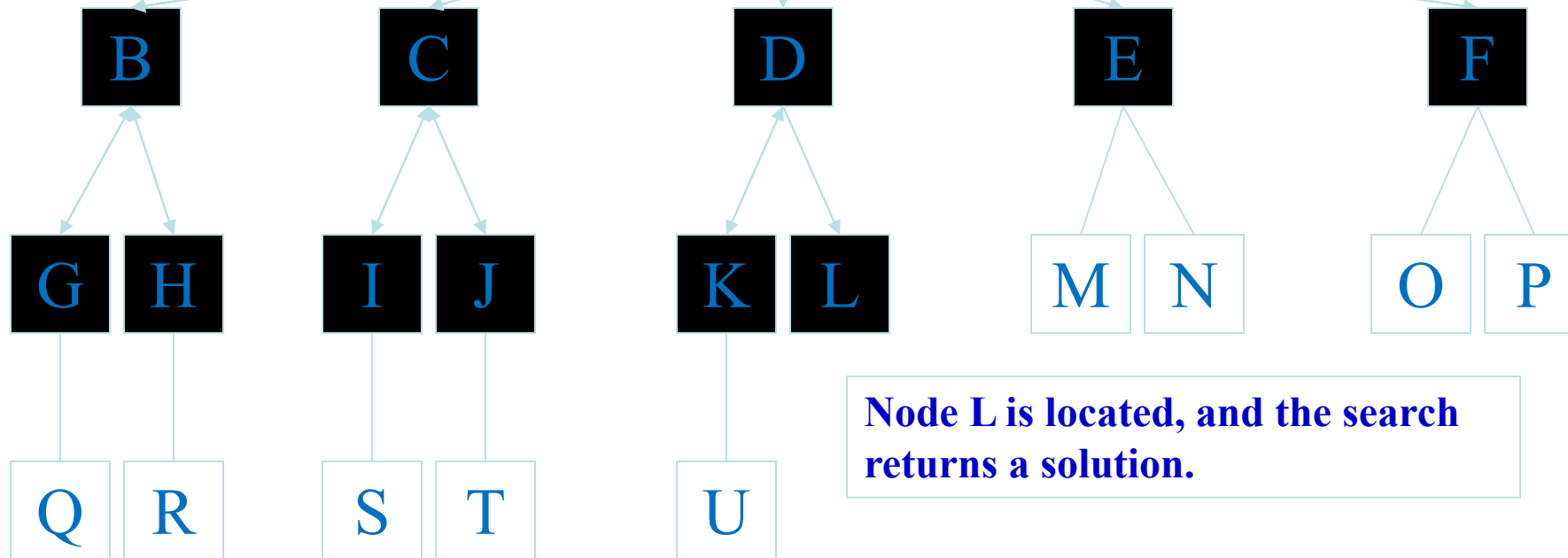
Three types of nodes during the search in tree search

- ▶ **Fringe nodes (open nodes, leaves)** in the queue
 - have been discovered
 - have not yet been processed
 - Child nodes not yet explored; not yet tested if they are goal
- ▶ **Visited nodes (closed nodes)**
 - have been processed
 - Nodes explored; tested if they match a goal
- ▶ **Undiscovered nodes**
 - have not yet been discovered



When **Enrich** finished the parallel divide and conquer, the queue of child nodes added to the END of Queue.

While going into the end of a line that represents the End of Queue (marked padded) has END of Queue.



Press space for the next step

Size of Queue: 0

Queue: Empty

Nodes expanded: 11

FINISHED SEARCH

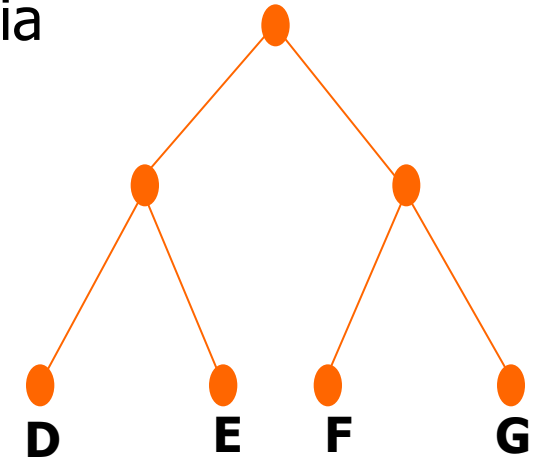
Current level: 2

Breadth-First Search Pattern

BFS - Evaluation

All the tree search techniques can be evaluated w.r.t 4 criteria

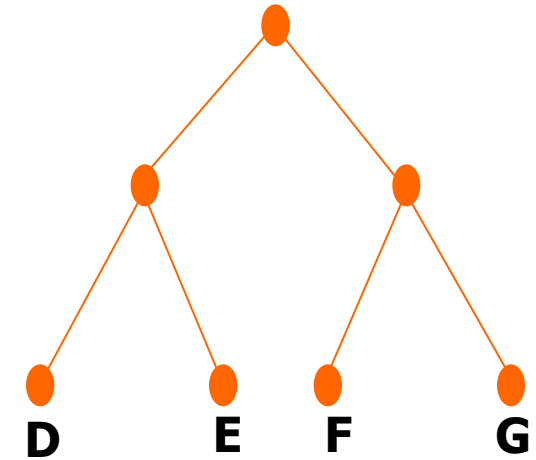
1. **Completeness** Yes
 - Guaranteed to find a solution if one exist?
2. **Time Complexity**
 - How long does it take to find a solution?
3. **Space Complexity**
 - How much memory is required to perform the search?
4. **Optimality** Yes
 - Find the optimal solution (one or all optimal solutions)?



BFS - Evaluation

- ▶ Evaluating against **four criteria**
 - Time Complexity
 - $O(b^d)$ i.e. no. of nodes in the tree in the worst case*
 - Space Complexity
 - $O(b^d)$ i.e. no. of nodes in the tree

b : the average branching factor
 d : is the depth of the search tree
- ▶ Big O*: notation in complexity theory. How the problem size affects the algorithm's computational resource (time or memory)
- ▶ **Exponential along with d**



Note:

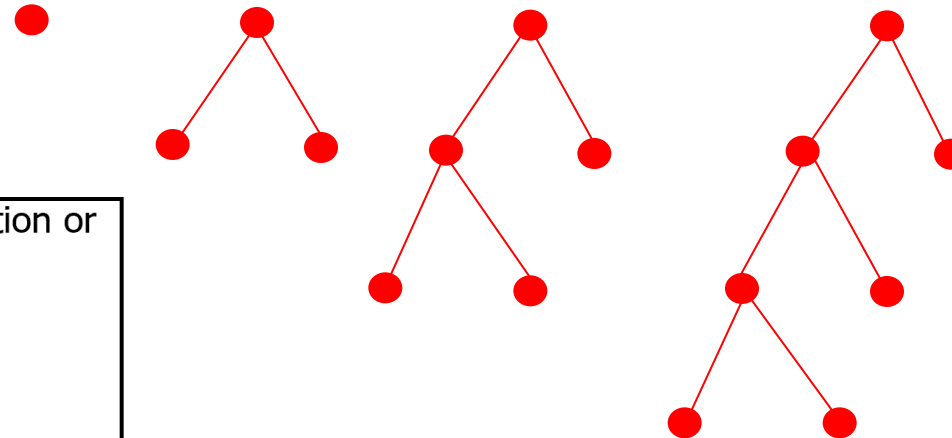
* The space / time could be less as a solution may be found before the d^{th} level

Depth First Search (DFS) - Method

- ▶ Expand Root Node First
- ▶ Explore one branch before exploring another branch
- ▶ Queuing function: adds nodes to the **front** of the queue

Function DFS(problem) returns a solution or failure

Return General-Search(problem, ENQUEUE-AT-FRONT)



- ▶ Function General-Search(p, QUEUING-FN) returns a solution or failure
 - **nodes** = Make-Queue(Make-Node(Initial-State[p]))
 - Loop do
 - If **nodes** is empty then return failure
 - **node** = Remove-Front(**nodes**)
 - If Goal-Test[p] on State(node) succeeds then return node
 - **nodes** = QUEUING-FN(**nodes**, Expand(node, Operators[p]))
 - End
- ▶ End Function

DFS – Evaluation

- ▶ **Completeness**
 - Infinite branch: never terminate if no goal state exist in that branch
- ▶ **Optimality**
 - Finds a solution: is there a better solution at a lower level?
- ▶ **Space complexity**
 - Store: the path from the root to the leaf node and the unexpanded neighbour nodes
 - DFS requires storage of bm nodes
 b : a branching factor; m : maximum depth
- ▶ **Time complexity**
 - b^m in the worst case
- ▶ DFS: neither complete nor optimal

1	4	7
2	5	8
3	6	

